Aajoo Homes — UAT Developer Setup

Developer setup for the controlled UAT run — every record section 4 of the Execution Pack asks for, the defects found while preparing them, and what remains open.

Aajoo Homes — BotPenguin controlled UAT
Prepared 5 September 2026 by the Zyphex Tech development team
Verified against the live development backend

1. Where things stand

Section 4 of the Execution Pack lists eleven items under *"Sumit: development portal and backend test records"*. **All eleven are ready.**

Preparing them surfaced four defects in the chatbot backend. Each would have produced a Fail on a specific case whatever the test data looked like, so all four were fixed and deployed before the run rather than left to be discovered during it.

Two items remain open. Neither is development work: both sit with Aajoo and the BotPenguin operator account, and each blocks one named case.

Checkpoint	Status
Development records ready	**11 of 11**
Defects fixed and deployed	**4**
Open, and not development work	**2**
Android build carrying these changes	**1.0.0 (23)**
Backend	**Deployed and confirmed serving the new code**

Every record below was checked against the live development backend and its database rather than reported from working notes. That matters here: two of the eleven had previously been recorded as satisfied and were not, so please work from this document rather than any earlier list.

2. The eleven records, one by one

1. Bot connected to the development website and app

Both surfaces carry the finalised **Aajoo Homes Mobile & Web** bot, ID `69803a093817049868bf064f`, window `696f4cdf88f4a8046c67188e` — the IDs named in section 2 of the pack.

On the website the bot is loaded on the public pages through the widget script. In the Android app it opens the hosted chat window at the same bot and window IDs.

2. The signed-in session is passed to the bot

Both surfaces now hand the bot an authenticated context when the user is already signed in, so the phone-capture step is skipped. **The tester never sees, copies or types a token**, as the pack requires.

What is passed is a **handoff token, not the login session**. It is minted per chat opening, lives fifteen minutes, is valid for starting a chat session and nothing else, and is signed with a key that the ordinary session verifiers reject. If it cannot be minted, the chat opens with no identity at all rather than falling back to the login session — the bot then asks for a phone number, which is the correct outcome.

> **This was the app's outstanding half.** Until 5 September the Android app put the user's thirty-day login credential into the chat URL, where it would have been recorded in BotPenguin's request logs and analytics. The website had already moved to handoff tokens; the app now matches it. Shipped in build 23.

3. Guest account with at least two non-cancelled bookings

`aajoo.renter1@mailinator.com` — display name *lala Tester*.

It currently holds **eighteen** non-cancelled bookings, so UAT-07's deterministic-primary-booking check has a real set to choose from.

4. Guest account with a stay in progress

Booking `B170046` on the same guest account. 5 to 8 September 2026, pay-at-property, status *Booking Confirmed*.

No account on the platform had a stay in progress, so this was created. It was also approved by the host after creation: a new booking sits at *Booked*, which is a request awaiting the host, and while the bot would have called it ongoing regardless, the host's own Ongoing Stays screen would not have listed it. A stay that two screens disagree about is not a usable test record.

> **This record expires.** The stay ends at 11am on 8 September, after which it stops counting as ongoing. If the run is still going, tell us and we will create a replacement. It has to be done in the afternoon — a booking only reads as ongoing once the 2pm check-in has passed, so one created in the morning still looks like an upcoming stay.

5. Host account with listing, analytics and payout data

`aajoo.host1@mailinator.com` — display name *Aajoo Test Host*.

Holds 29,261 listings and 61 bookings as host, with payout history behind them, so listing search, analytics and payout answers all return real data.

6. One account that is both a Guest and a Host

`aishmobilehost@yopmail.com` — display name *Aish Mobile Host new*. Four listings, three of them live.

No account on the platform held both roles; every one was a host or a guest and never both. This account now holds both.

> **Sign in on the Host tab for this account.** Aajoo's sign-in screen asks which side you are signing in on, and this account's credentials are registered on the host side. Once signed in, open chat and choose Guest or Host there — which is what UAT-05 describes. The platform also exposes a mode switch for moving between the two once signed in.

Note for UAT-08 and the cross-host test: because this account is now the dual-role one, prerequisite 8's second host is a **different** account (below), so the two cases never read the same record.

7. A booking owned by somebody else

Booking `B754668`, owned by a different development guest account (*Aishwary testing*), for the ownership-denial test.

8. A second Host for the cross-host denial test

`itsme.ashsriv007+host2@gmail.com` — display name *Aish test Host*. Two listings.

Distinct from both the main host in item 5 and the dual-role account in item 6.

9. A disposable booking that may be cancelled exactly once

Booking `B261280` on the guest account. 19 to 21 September 2026, pay-online, currently unpaid.

Please use this booking for UAT-16 and UAT-17 and for nothing else. Every other future booking on that guest is counted by prerequisite 3 and is part of the set UAT-07 picks its primary from; cancelling one of those mid-run would change what UAT-07 should expect.

It is deliberately a pay-online booking rather than pay-at-property. It stays unpaid, so cancelling it moves no money and leaves no refund to unwind — which keeps the money side out of the run, as the pack intends.

10. Which inbox receives which OTP

OTP	Delivered to
Guest cancellation — UAT-17	`aajoo.renter1@mailinator.com`
Host payout — UAT-36	`aajoo.host1@mailinator.com`

The code goes to the account's registered email address, never to the phone number used in the chat. One code path serves every OTP the bot sends, whatever the action, and addresses it to whichever account the session has resolved. There is no separate payout inbox.

11. WhatsApp sender mapped to each role

Role	WhatsApp sender	Account
Guest	`9882498033`	`aajoo.renter1@mailinator.com`
Host	`9611577338`	`aajoo.host1@mailinator.com`

These two mappings are stable for the run.

> **A duplicate account had been quietly breaking this.** A second live account had been created on the Guest number and flagged as a host, so the same handset resolved to the test guest in a Guest session and to a different person's account in a Host session — exactly the state the pack warns against. A Host payout OTP started from that number would have gone to the wrong inbox, contradicting the answer given for item 10.
>
> This is resolved. Each number now resolves to exactly one account under every role: the Guest number to the test guest and to nobody in a Host session, the Host number to the test host and to nobody in a Guest session. Nothing had been mis-resolved before the fix — every chatbot session ever recorded on that number had resolved correctly.

---

3. Defects found while preparing the records

All four are fixed, deployed, and confirmed live.

The app was handing the chat a login credential

Relates to prerequisite 2, and to UAT-03, UAT-04 and UAT-05.

Covered in item 2 above. Fixed in Android build 23.

A dual-role account was always treated as a Host

Fails UAT-05.

The bot worked out which role a session was in from the account's stored flags rather than from the role the person chose in the chat, and Host won that test. Because no account had ever held both roles, the flag and the choice always agreed and the fault was invisible — until item 6 created an account that holds both. Choosing Guest returned Host.

The chosen role now wins whenever the account genuinely holds it. A role the account does *not* hold is still corrected, which is what the old test was protecting.

The same change closed a second gap that item 6 would have exposed: the session context carried the account's own guest bookings whatever role the session was in. Harmless while no host had ever booked a stay, but a dual-role account would have had Host mode answering with that person's own trips — which is the leak UAT-05's *"Guest data is not shown in Host mode"* looks for. A Host session no longer carries them.

Cancelling a booking twice reported success twice

Fails the final step of UAT-17.

Nothing checked what state a booking was in before cancelling it. Cancelling an already-cancelled booking set it to cancelled again and answered *"your booking has been cancelled successfully"* — which is precisely UAT-17's last step, whose pass condition is that the repeat terminates safely instead of reporting a second cancellation.

The repeat is now refused with a clear message, and so is cancelling a stay that has already started. Three further things the same path had been skipping were added at the same time: it now records **when** a booking was cancelled and **under which policy**, it releases the host from the commission charge the booking had raised, and it quotes the refund from the property's actual cancellation policy. Previously it used a rule of its own that matched none of the published policies.

Verified by driving a real cancellation twice on a throwaway booking: the first succeeded, the second returned a refusal.

Every one-time code was written to the server log

Relates to prerequisite 10 and to the pack's result-logging rule.

The chatbot wrote each OTP into the server log in clear text, beside the address it was sent to, under a code comment claiming the line had been removed. It had not. Those codes sat in the log for as long as logs are kept, readable by anyone with dashboard access, long after the code itself expired.

The pack asks testers never to place a full OTP in a result log, group chat or recording. The service was doing it on their behalf, for every code they were about to generate.

The log now records only that a code went out, the session it belongs to, and the recipient masked to its first letter and its domain — so a wrong-inbox problem stays diagnosable without the address being written down.

---

4. Quick reference for the run

Accounts

Role	Sign in with	WhatsApp sender
Guest	`aajoo.renter1@mailinator.com`	`9882498033`
Host	`aajoo.host1@mailinator.com`	`9611577338`
Guest **and** Host	`aishmobilehost@yopmail.com` — *Host tab*	not mapped
Second Host	`itsme.ashsriv007+host2@gmail.com`	not mapped

Sign-in credentials are not printed in this document. They will be shared separately through the agreed channel, in line with the pack's rule that passwords and tokens are never placed in a written record.

Bookings

Booking	Use it for	Dates	State
`B170046`	The stay in progress	5–8 Sep 2026	Booking Confirmed, pay-at-property
`B261280`	The one cancellation test, and nothing else	19–21 Sep 2026	Unpaid, pay-online
`B754668`	The ownership-denial test	—	Owned by a different guest

Do not do these

- **Do not map the same WhatsApp sender to both roles.** The two numbers above are separate on purpose, and a duplicate account that broke this has just been cleared. If only one tester handset is available, tell us before remapping so we can confirm the change and you can reset the old conversation first.
- **Do not spend a booking other than `B261280` on a cancellation test.** The others are load-bearing for prerequisites 3 and 4 and for UAT-07.
- **Do not use the public Incognito link for UAT-03, UAT-04 or UAT-05.** Those three prove that a signed-in Aajoo session reaches the bot, so they have to be run from inside the signed-in development website or app.

---

5. Open items — not development work

Both of these sit with Aajoo and the BotPenguin operator account, and each blocks one named case.

A second BotPenguin operator — UAT-29

The transfer sub-test needs a second eligible operator in the **Aajoo Support Team**. The live account currently shows a single team member, so a second user has to be added or activated before a transfer can be accepted as a pass.

Confirm a real recipient received the alert — UAT-31

The email recipients saved in the target bot's Alerts settings need checking, and at least one of them has to confirm the fresh email actually arrived. Addresses appearing in settings is not the test.

---

6. Known behaviours worth knowing before you start

A pending OTP does not survive a backend restart. Codes awaiting verification are held in the running process, so a deploy or a restart expires all of them at once. A tester who requests a code and enters it after one will be told it expired. Expect this at least once across a week-long run — it is not a fault in the verification logic, and requesting a fresh code is the whole remedy. We have deliberately left this alone: moving it to stored state is a change worth making on its own terms, not the day before a UAT run.

A stay counts as ongoing from 2pm on the arrival day, and stops counting at 11am on the departure day. This governs which booking the bot treats as the current stay.

Cancelled bookings are excluded from the booking list the bot offers. A guest choosing which booking they are asking about is not offered a stay that is no longer happening.

---

7. How this was verified

Every one of the eleven records was checked against the live development backend and its database, not against working notes. The audit is a script we can re-run on request, so the same list can be confirmed again at any point during the run.

The four fixes carry automated tests — eighteen in total across the two areas — and the backend test suite passes in full. The two behavioural fixes were additionally driven against the live system: the cancellation was performed twice on a throwaway booking to confirm the repeat is refused, and a real one-time code was issued to confirm the log no longer contains it while the email still arrives.

The backend was confirmed serving the new code after release, and the Android build carrying the app-side change is 1.0.0 (23).

We are happy to walk through any of this before testing begins, and to prepare any further development record on request.